AuraCards

Document Based Flashcard Generation and Focused Study

Nandan P R – 23MIC0065

github link: https://github.com/thanosisinevitable/Auracards

AuraCards turns study documents into editable flashcards and gives learners a place to practise what they remember. This writeup explains why the project was built, how its parts fit together, and what happens between uploading notes and starting a study session. It also covers the Focus companion, which connects AuraCards to the existing Pomodoro website.

Motivation

A folder full of notes can feel productive. Remembering what is in those notes is the harder part. Students often revise from a mix of PDFs, Word documents and presentation slides, then spend more time turning that material into questions. Switching to a separate timer adds another small interruption.

AuraCards was built to make that routine easier. Upload the notes, check the suggested cards and start practising in the same workspace. The aim is to spend less time preparing revision material and more time actually using it. The review step matters because a generated card still needs a human eye before it deserves a place in a revision deck.

Existing Gap and Unique Selling Proposition

The gap AuraCards addresses is the work between having notes and being ready to revise. A document viewer shows the material, a flashcard tool helps practise it, and a timer manages the session. Putting these steps together reduces the setup and switching that can eat into a short study break.

AuraCards offers a notes-to-focus study workspace: turn existing documents into editable cards, choose how those cards are generated, and connect a deck to a live focus timer without leaving the study screen. Its selling point is this complete revision loop, with the learner in control of the content and the pace.

That control has practical value. Offline rules can turn structured notes into cards without a paid AI API. Optional local or cloud AI provides another generation route when configured. Source information helps with checking, cards remain editable, and manually linking a deck makes the study session follow the focus interval. The notes do not just sit in a folder looking important; they become something the learner can work through.

Architecture

AuraCards has two main parts: the interface in the browser and a Python backend. The browser handles decks, editing and studying. The backend processes uploaded files, generates draft cards and communicates with connected services. Waitress receives web requests and hands them to Flask, which runs the matching application route. [1][2]

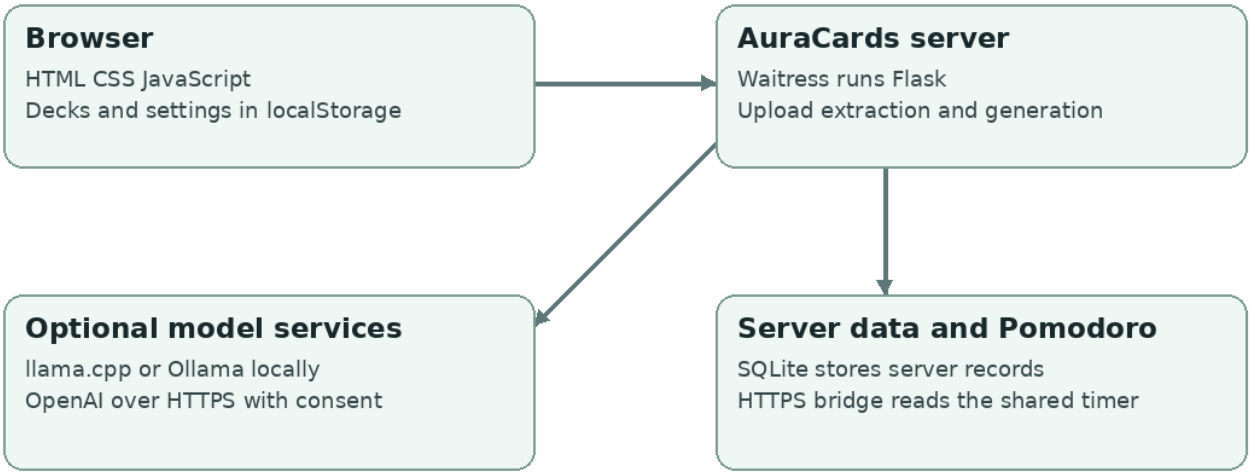


Figure 1 Main components and their connections

Component	Responsibility	Main project files
Browser interface	Upload forms, review editor, decks, study screens and themes	dist/index.html dist/app.js dist/styles.css dist/themes.css
Python web layer	Serve the frontend, validate requests and expose API routes	app.py
Document processing	Extract readable text and keep source locations	backend/extraction.py
Card generation	Choose a generation route, build cards and check results	backend/generation.py backend/smart.py backend/structures.py
Focus connection	Display the companion and proxy timer requests	dist/linked-study.js backend/integration.py backend/local_bridge.py

The table shows where the main responsibilities sit in the project. Keeping these jobs separate makes it easier to change the interface without rewriting document extraction, or improve generation without rebuilding the study screen.

How the generation workflow operates



Figure 2 From uploaded notes to a study session

The process starts with a file, a deck name and a few generation choices. The backend checks the request, selects the right parser and cleans the extracted text. It also keeps useful source details, such as a PDF page or slide location, so a card can be checked against the material it came from.

Offline generation

The offline generator first looks for structure that is already useful. Question and answer pairs can become cards directly, while a topic heading and its explanation can provide another starting point. For ordinary prose, rules select facts, build questions and rank the results. Duplicate or unsuitable candidates are filtered. This route runs without a language model, so it does not need an AI API key.

Optional AI generation

With llama.cpp, the backend finds a running model server and sends selected sections of the notes along with instructions. That source text is how the model gets the information for its cards. Longer documents are handled in smaller source windows to stay within the model context limit. The model does not independently browse the uploaded file or learn it permanently. [8]

Balanced and Thorough modes can plan topics, draft cards and review them against the selected source. Flash takes a shorter drafting route when a quick first pass is more useful. Their model-work budgets are approximately 8, 15 and 3 minutes respectively. These are processing limits, not a promise that every deck takes that long. A compatible loaded model can be used, so the application is not tied to one model.

Cloud generation sends extracted text to OpenAI after user consent and requires a server API key. Ollama offers another optional local route. Results from every method return to the review screen. Progress updates show the stages being completed. [9][10]

Review and study

The user checks the draft, edits anything unclear and saves the deck. Studying then follows a familiar pattern: try the question, flip the card and check the answer. Again makes the card due after about 10 minutes; Got it makes it due after about one day. These are simple fixed intervals, rather than an adaptive scheduling algorithm. No points for recognising an answer after flipping too early.

Complete Techstack Information

The stack below covers AuraCards and its connected timer. Each tool has a specific job, from drawing the interface to extracting text. Versions refer to the project dependency pins; they are not a list of the newest releases.

Technology	Role and reason for use
HTML5	Defines the upload form, navigation, dialogs and study screens. Native browser elements keep the interface straightforward.
CSS3	Controls layouts, responsive behaviour, themes and card effects. Aurora uses mint and lavender accents with dark panels.
Vanilla JavaScript	Manages browser state, editing, card flips, events and API calls. This interface does not require a separate frontend framework.
Fetch API and JSON	Move requests and structured data between the browser and backend. Multipart form data carries document uploads.
Python 3	Implements extraction, generation, validation, storage and service integration in one backend language.
Flask 3.1.3	Maps URLs to Python handlers and provides request, response and session support. [1]
Waitress 3.0.2	Runs the Flask WSGI application as an HTTP service. It handles serving requests while Flask handles application behaviour. [2]
pypdf 6.10.0	Extracts text from PDF pages. Scanned image pages require OCR, which this project does not include. [3]
python-docx 1.2.0	Reads Word document content, including paragraphs and tables. [4]
python-pptx 1.0.2	Reads PowerPoint slide content, including supported text and table structures. [5]
Python standard library	sqlite3 stores data; json encodes records; urllib makes HTTP calls; threading and queue support background work; multiprocessing isolates extraction.

Werkzeug supports HTTP request handling as part of Flask. The main interface is built with browser JavaScript, so most screen changes happen without generating a new page on the server. [1]

Storage models and deployment tools

Technology	Role and reason for use
Browser localStorage	Persists categories, decks, cards, preferences and review data on that browser origin. It keeps basic study operations local to the user. [6]
Browser sessionStorage	Keeps temporary study-position information for the current browser session. It is separate from the long-term deck store.
SQLite	Stores server-side focus records and shared-library snapshots. Pomodoro stores its timer and related application records in SQLite. A file-based database suits this small deployment. [7]
llama.cpp and GGUF models	Optional local inference runtime and model format. A running compatible model server is needed; model weights are not built into AuraCards. [8]
Ollama	Optional alternative local model service using its chat API. [9]
OpenAI Responses API	Optional cloud generation service. The reviewed code uses a configurable cloud model, with gpt-4.1-mini as its default. [10]
Git and GitHub	Track source changes and provide the repository that Render deploys.
Render	Hosts the Python web service and supplies its public URL and listening port. [11]
Pomodoro Python service	Uses Python HTTP handling and SQLite to provide the shared timer API. Its optional Gemini assistant is a separate Pomodoro feature, not a dependency of the AuraCards Focus companion.

One hosting detail is easy to miss: localhost means the computer running the backend. On Render, it refers to the Render service, not the learner laptop. A model running only on that laptop therefore needs a local AuraCards setup or another supported server arrangement. Lightweight generation and a configured cloud provider are the practical choices for this hosted design.

Decks saved in browser storage belong to that browser and website origin. They do not automatically appear on another device, which is why JSON backup export is useful. Server SQLite files need persistent hosting storage to survive redeploys. Render free services use an ephemeral filesystem. [6][12]

Datasets Used

The uploaded notes are the source material for each generation request. AuraCards does not need an external training dataset, fine-tune a model or train a classifier. The documented pipeline also does not create embeddings. In everyday terms, it works from the notes supplied for that session rather than building a new AI model.

Development examples included Python study notes and structured Python notes in PDF, DOCX and PPTX formats. They were used to check extraction and card generation, not as a labelled benchmark. Training data behind any third-party model is separate from these project inputs. The project does not claim a measured accuracy score or a proven improvement in exam results.

Selected Development Prompts

ChatGPT was used as a development assistant. The examples below are shortened versions of selected requests about card quality, portability and the study experience. They show the main requirements being refined rather than a complete conversation log.

1 Improve card quality. Recognise notes written as question and answer pairs or topics followed by explanations, and improve the framing of generated questions.

2 Keep offline use practical. Make the application straightforward to run offline on different computers, especially Ubuntu, without requiring Docker.

3 Make focused study feel connected. Add an adjustable Pomodoro companion beside the cards, match the existing website theme, and pause linked studying when the timer pauses or enters a break.

4 Give the learner control. Make deck linking a manual choice so users can decide when their cards should follow the timer.

API Integration

The browser sends requests to the AuraCards API on its own website origin. Flask validates the input and returns JSON or generation progress updates. Requests that change browser-controlled state use an Aura CSRF token. For the Focus companion, the Python backend makes the call to Pomodoro and returns the timer state to the browser.

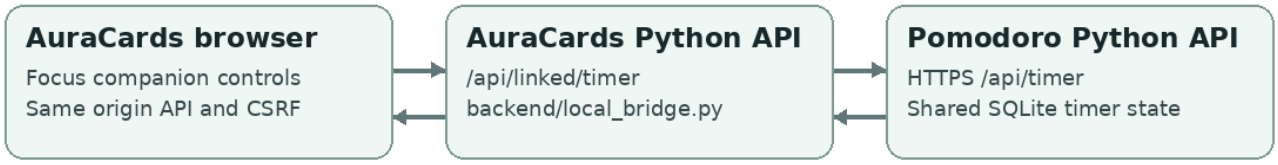


Figure 3 Focus companion requests and timer responses

API route	Purpose and data
GET /api/config	Returns frontend settings and the CSRF token for the browser session.
POST /api/generate	Accepts a multipart document upload and generation options; returns draft cards or progress events.
GET /api/linked/timer	Reads the shared Pomodoro timer through the AuraCards backend.
POST /api/linked/timer	Sends start, pause, reset, mode, settings or deck commands with the latest timer revision.
Pomodoro /api/timer	The existing timer endpoint receives server requests and stores the authoritative timer state in SQLite.
POST /api/study-library	Stores a validated deck snapshot on the AuraCards server. This does not replace the browser deck store.
GET /api/health	Provides a simple service health response.

The hosted connection sets POMODORO_URL to https://pomodoro-website-python-project.onrender.com. AuraCards calls the existing /api/timer endpoint over verified HTTPS, so this version can work without editing Pomodoro. It requires a compatible deployed timer API. No extra STUDY_BRIDGE_TOKEN is needed for this connection.

Timer responses include the mode, running state, remaining time, settings and revision. Commands carry the current revision. If another client has already changed the timer, HTTP 409 signals the conflict and the client can fetch the newer state. Between updates, the browser keeps the countdown moving using the server snapshot and a monotonic clock.

AI services use separate connections. llama.cpp provides a compatible local model API, Ollama uses /api/chat, and cloud generation uses the OpenAI Responses API. Cloud credentials stay on the server. Starting a focus timer does not require an AI key. [8][9][10]

Users Inputs and Outputs

Users and their tasks

AuraCards is aimed at students revising course notes, independent learners working through reference material and teachers preparing draft revision cards. A typical session starts with a chapter or slide deck and ends with a reviewed set of cards to practise. The everyday browser workflow does not require programming knowledge.

Input	Accepted form and relevant limits
Study documents	PDF (.pdf), Word (.docx) and PowerPoint (.pptx). The reviewed upload limit is 20 MiB per document.
Generation options	Deck name, category, count from 1 to 50, difficulty, card style, input format, generation method and optional focus text.
Structured notes	Question and answer pairs or topic headings followed by explanations. General prose is also accepted.
Manual changes	Edits to questions and answers, category choices, starred cards and recall ratings.
Timer controls	Focus and break durations, start or pause, reset, mode and optional deck linking.
Backup and service settings	JSON deck backups and valid configured service URLs. Local AI also needs a running compatible model.

Outputs and user control

Generation produces draft question and answer cards, including ordinary questions and fill-in-the-blank formats. Source information can help the user check the material, and the review screen allows edits before saving. The finished deck can be browsed and studied in the browser, while JSON export provides a portable backup.

During study, the question appears first and the answer is revealed on a flip. Again and Got it record the recall rating. The Focus companion adds a timer alongside this workflow. If a deck is manually linked, its study controls pause during breaks or when the timer is paused, then unlock during an active focus interval.

Practical limits

Scanned image-only PDFs need OCR, which is not included. Complex layouts, equations and messy notes can also produce incomplete text or weak cards. Automated checks help filter problems, but they cannot guarantee a correct answer every time. The review screen is there for a reason.

The connected Pomodoro implementation uses one shared timer per server rather than separate account timers. Browser storage also has limits: clearing site data can remove decks, and changing the website origin creates a separate store. Keeping a backup is a small habit that can save a fairly large headache.

References

The following official documentation supports the descriptions of the technologies used. Links were consulted on 31 October 2026.

[1] Pallets Projects. Flask documentation. https://flask.palletsprojects.com/en/stable/

[2] Pylons Project. Waitress documentation. https://docs.pylonsproject.org/projects/waitress/en/stable/

[3] pypdf project. Text extraction documentation. https://pypdf.readthedocs.io/en/stable/user/extract-text.html

[4] python docx project. Library documentation. https://python-docx.readthedocs.io/en/latest/

[5] python pptx project. Library documentation. https://python-pptx.readthedocs.io/en/latest/

[6] MDN Web Docs. Window localStorage reference. https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage

[7] SQLite project. About SQLite. https://www.sqlite.org/about.html

[8] ggml org. llama.cpp repository and documentation. https://github.com/ggml-org/llama.cpp

[9] Ollama. Chat API documentation. https://docs.ollama.com/api/chat

[10] OpenAI. Text generation and Responses API guide. https://developers.openai.com/api/docs/guides/text

[11] Render. Web services documentation. https://render.com/docs/web-services

[12] Render. Free service limits and storage behaviour. https://render.com/docs/free

Project deployment addresses

AuraCards: https://auracards.onrender.com/
Pomodoro: https://pomodoro-website-python-project.onrender.com/

github link: https://github.com/thanosisinevitable/Auracards